

Contents

1	Overview	3
1.1	Key Specifications	3
1.2	Data Format	3
2	Modbus Register Map	4
2.1	Input Registers – FC04 (Read-Only)	4
2.1.1	Status & Metadata (0x0000–0x0005)	4
2.1.2	Latest Sample (0x0010–0x001B)	4
2.1.3	Bulk Data (0x0100+)	4
2.2	Holding Registers – FC03/FC06 (Read-Write)	6
2.2.1	Command Codes (CMD register 0x0000, write via FC06)	6
2.2.2	Full-Scale Register (FS, 0x0002)	6
2.2.3	ODR Register (ODR, 0x0003)	6
2.2.4	Configuration Sequence	6
3	Python Acquisition Script	7
3.1	Requirements	7
3.2	Usage	7
3.2.1	Command-Line Options	7
3.3	Test Mode	7
3.4	Stream Mode	8
3.5	CSV Output Format	8
3.6	USB Latency	8
4	Communication Parameters	9
4.1	Throughput	9
5	Verification	10
5.1	Sample Rate Measurement	10
5.1.1	Test Setup	10
5.1.2	Test Command	10
5.1.3	Results	10
5.1.4	Rate Stability	10
5.1.5	LIS2DS12 Oscillator Characteristics	11
6	Hardware	12
6.1	Wiring	12
6.2	P100 Connector Pinout	12
6.2.1	Pin 1 Identification	12
6.3	Connection Diagram	13
6.4	Wiring Guidelines	13
6.5	Status LED	13
6.6	Boot Sequence	13

7	Firmware Update	14
7.1	Requirements	14
7.2	Update Procedure	14
7.2.1	Step 1: Connect	14
7.2.2	Step 2: Flash new firmware	14
7.2.3	Step 3: Verify	15
7.3	LED Indicators During Update	15
7.4	Troubleshooting	15
7.4.1	Update script reports “No response from bootloader”	15
7.4.2	Unit is unresponsive after a failed update	15
7.4.3	Validation fails	15

1 Overview

The AF10R1 is a 4-channel accelerometer module based on the STM32G0B1KEUx MCU. It reads four LIS2DS12 MEMS accelerometers at ~1600 Hz and exposes synchronized XYZ data over RS485 using Modbus RTU.

1.1 Key Specifications

Parameter	Value
MCU	Cortex-M0+, 64 MHz
Accelerometers	4x LIS2DS12
Sample rate	100–1600 Hz (configurable, default 1600 Hz)
Full scale	2g/4g/8g/16g (configurable, default 2g)
Interface	RS485, Modbus RTU, 921600 baud, 8N1
Ring buffer	1024 sample-sets (24 KB RAM)
TX method	DMA (non-blocking)
FIFO trigger	INT1 threshold at 32 samples per sensor
Status LED	RGB on PB6/PB7/PB8 (green=OK, red=fault)

1.2 Data Format

Each sample-set contains 12 values: XYZ for each of the 4 sensors. Values are raw int16 from the LIS2DS12. The conversion factor depends on the configured full-scale range:

Full Scale	Sensitivity (mg/LSB)	Z at rest (typical)
2g	0.061	~16384 LSB
4g	0.122	~8192 LSB
8g	0.244	~4096 LSB
16g	0.488	~2048 LSB

To convert a raw value to milligravity:

`mg = raw_value * sensitivity`

Example (2g mode): $16384 * 0.061 = 999 \text{ mg} \sim 1\text{g}$.

2 Modbus Register Map

Slave address: 1 (default, configurable via holding register 0x0001).

2.1 Input Registers – FC04 (Read-Only)

2.1.1 Status & Metadata (0x0000–0x0005)

Address	Name	Type	Description
0x0000	STATUS	uint16	Bit 0–3: sensor 1–4 OK (1=detected). Bit 8: sampling active
0x0001	AVAIL	uint16	Unread sample-sets in ring buffer
0x0002	OVERRUN	uint16	Samples lost due to buffer overflow
0x0003	ODR	uint16	Configured output data rate in Hz (e.g. 1600)
0x0004	COUNTER_L	uint16	Total sample counter, low word
0x0005	COUNTER_H	uint16	Total sample counter, high word

STATUS register bit layout:

Bit:	15..9	8	7..4	3	2	1	0
	reserved	sampling	reserved	LIS4	LIS3	LIS2	LIS1
		active		OK	OK	OK	OK

2.1.2 Latest Sample (0x0010–0x001B)

The most recent sample-set, updated continuously. Reading does not consume data from the ring buffer. Useful for spot-checking sensor output without starting a stream.

Address	Name	Content
0x0010	S1_X	Sensor 1, X-axis (int16)
0x0011	S1_Y	Sensor 1, Y-axis
0x0012	S1_Z	Sensor 1, Z-axis
0x0013	S2_X	Sensor 2, X-axis
0x0014	S2_Y	Sensor 2, Y-axis
0x0015	S2_Z	Sensor 2, Z-axis
0x0016	S3_X	Sensor 3, X-axis
0x0017	S3_Y	Sensor 3, Y-axis
0x0018	S3_Z	Sensor 3, Z-axis
0x0019	S4_X	Sensor 4, X-axis
0x001A	S4_Y	Sensor 4, Y-axis
0x001B	S4_Z	Sensor 4, Z-axis

2.1.3 Bulk Data (0x0100+)

Bulk read region for high-speed streaming. Each read consumes data from the ring buffer – sample-sets are returned once and then discarded.

Each sample-set occupies 12 consecutive registers:

Offset (within set)	Content
0	S1_X
1	S1_Y
2	S1_Z
3	S2_X
4	S2_Y
5	S2_Z
6	S3_X
7	S3_Y
8	S3_Z
9	S4_X
10	S4_Y
11	S4_Z

Maximum 84 sample-sets (1008 registers) per read. Request must be a multiple of 12 registers.
Example: to read 42 sample-sets, send FC04 with start=0x0100, quantity=504.

2.2 Holding Registers – FC03/FC06 (Read-Write)

Address	Name	Type	Description
0x0000	CMD	uint16	Command register (write triggers action)
0x0001	ADDR	uint16	Slave address (1–247, default 1)
0x0002	FS	uint16	Full-scale range: 0=2g, 1=4g, 2=8g, 3=16g (default 0)
0x0003	ODR	uint16	Output data rate in Hz (default 1600). See table below

2.2.1 Command Codes (CMD register 0x0000, write via FC06)

Value	Name	Description
1	START	Start sampling. Flushes sensor FIFOs, clears interrupts, activates ring buffer writes
2	STOP	Stop sampling. Ring buffer retains unread data
3	RESET_BUF	Clear ring buffer and overrun counter. Does not affect sampling state

2.2.2 Full-Scale Register (FS, 0x0002)

Write before starting sampling. Takes effect on the next START command.

Value	Range	Sensitivity (mg/LSB)
0	2g	0.061
1	4g	0.122
2	8g	0.244
3	16g	0.488

2.2.3 ODR Register (ODR, 0x0003)

Write before starting sampling. Takes effect on the next START command. Valid values:

Value	Data rate
100	100 Hz
200	200 Hz
400	400 Hz
800	800 Hz
1600	1600 Hz

2.2.4 Configuration Sequence

FS and ODR must be written while sampling is stopped. Typical sequence:

1. Write FS register (FC06, address 0x0002).
2. Write ODR register (FC06, address 0x0003).
3. Write CMD=START (FC06, address 0x0000, value 1).

3 Python Acquisition Script

The script `test_modbus.py` provides hardware verification tests and continuous streaming to CSV.

3.1 Requirements

```
pip install minimalmodbus pyserial
```

3.2 Usage

```
# Run hardware verification tests (default: 2g, 1600 Hz)
python test_modbus.py /dev/ttyUSB0

# Test with specific full-scale and ODR
python test_modbus.py /dev/ttyUSB0 --fs 4g --odr 800

# Stream to auto-named CSV (accel_YYYYMMDD_HHMMSS.csv)
python test_modbus.py /dev/ttyUSB0 stream

# Stream with 8g full-scale and 800 Hz ODR
python test_modbus.py /dev/ttyUSB0 stream --fs 8g --odr 800

# Stream to specific file with configuration
python test_modbus.py /dev/ttyUSB0 stream data.csv --fs 16g --odr 400
```

3.2.1 Command-Line Options

Option	Values	Default	Description
<code>port</code>	serial device path	–	Required. E.g. <code>/dev/ttyUSB0</code>
<code>command</code>	<code>test</code> , <code>stream</code>	<code>test</code>	Test mode or streaming mode
<code>csv</code>	file path	<code>auto</code>	CSV output path (stream mode only)
<code>--fs</code>	<code>2g</code> , <code>4g</code> , <code>8g</code> , <code>16g</code>	<code>2g</code>	Accelerometer full-scale range
<code>--odr</code>	<code>100</code> , <code>200</code> , <code>400</code> , <code>800</code> , <code>1600</code>	<code>1600</code>	Output data rate in Hz

3.3 Test Mode

Configures FS and ODR registers before running 7 sequential tests:

Test	Name	Description
1	Status	Read STATUS, verify sensor OK bits and ODR
2	Start sampling	Send START command, verify sampling bit set
3	Data flow	Wait 500 ms, verify AVAIL > 0 and COUNTER (threshold adapts to configured ODR)
4	Latest sample	Read 0x0010–0x001B, check plausible XYZ
5	Bulk read	Read 10 sample-sets from 0x0100, verify consumed
6	Overrun check	Verify OVERRUN == 0
7	Stop sampling	Send STOP command, verify sampling bit cleared

3.4 Stream Mode

1. Reads STATUS to verify sensors are online.
2. Writes FS and ODR holding registers (per `--fs` and `--odr` flags).
3. Sets USB-serial adapter to low-latency mode (`ASYNC_LOW_LATENCY` ioctl or sysfs).
4. Sends RESET_BUF then START commands.
5. Enters streaming loop:
 - Polls AVAIL register.
 - When data is available, reads `min(avail, 84)` sample-sets via raw FC04 (bypasses minimalmodbus 125-register limit).
 - Writes rows to CSV.
 - Checks OVERRUN every 10th iteration.
 - Prints live status line with sample count, rate, overruns, and error counters.
6. On Ctrl+C: sends STOP command, prints summary.

3.5 CSV Output Format

The CSV header comment reflects the configured full-scale range and sensitivity:

```
# LIS2DS12 4g mode, 0.122 mg/LSB, raw int16 values
sample,time,s1_x,s1_y,s1_z,s2_x,s2_y,s2_z,s3_x,s3_y,s3_z,s4_x,s4_y,s4_z
0,0.0234,-42,128,8192,-38,135,8201,-45,121,8178,-40,130,8190
1,0.0234,-41,129,8193,-37,134,8200,-44,122,8179,-39,131,8191
...
```

Column	Description
sample	Sequential sample number (0-based)
time	Seconds since stream start (Python clock)
s1_x..s4_z	Raw int16 accelerometer values

To convert a raw value to mg, use the sensitivity from the CSV header comment (depends on full-scale setting).

3.6 USB Latency

USB-serial adapters (FTDI, CH340) buffer data with a default 16 ms latency timer. This severely limits throughput. The script attempts to set low latency automatically. If it fails:

```
echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

4 Communication Parameters

Parameter	Value
Baud rate	921600
Data bits	8
Parity	None
Stop bits	1
Protocol	Modbus RTU
Slave addr	1 (default)
RS485 DE	PA12 (hardware-managed)
Framing	IDLE line detection
Max FC04	1008 registers (84 sets)

Frame detection uses USART IDLE line interrupt instead of the standard Modbus 3.5-character timer. At 921600 baud, 3.5 characters = ~38 us which is impractical to time precisely on a Cortex-M0+. The IDLE detector provides equivalent end-of-frame signaling.

4.1 Throughput

Throughput depends on the configured ODR. Measured values at 1600 Hz (default):

Metric	Value
Sensor ODR (nominal)	1600 Hz (configurable 100–1600 Hz)
Sensor ODR (actual)	~1493 Hz (see verification below)
Measured throughput	~1493 samples/s, 0 overruns
Data per sample-set	24 bytes (12x int16)
Raw data rate	~35.8 KB/s

5 Verification

5.1 Sample Rate Measurement

A 5-minute continuous streaming test was performed to verify the actual sample rate and data integrity.

5.1.1 Test Setup

Parameter	Value
Duration	300.4 seconds (5 minutes)
Mode	Continuous streaming via <code>test_modbus.py</code>
Interface	RS485 USB adapter (FTDI), 921600 baud
Host	Linux PC, Python 3

5.1.2 Test Command

```
python test_modbus.py /dev/ttyUSB0 stream
```

5.1.3 Results

Metric	Value
Total samples collected	448 517
Total duration	300.4 s
Average sample rate	1 493 Hz
Overruns (lost samples)	0
Short frame errors	0
CRC errors	0
Nominal ODR (configured)	1 600 Hz
Deviation from nominal	-6.7%

5.1.4 Rate Stability

The sample rate was analyzed across 10 equal time windows (30 seconds each):

Window	Time (s)	Sample rate (Hz)
1	0 – 30	1500
2	30 – 60	1492
3	60 – 90	1492
4	90 – 120	1492
5	120 – 150	1493
6	150 – 180	1492
7	180 – 210	1493
8	210 – 240	1492
9	240 – 270	1493
10	270 – 300	1493

The rate is stable at ~1 493 Hz throughout the test with no drift or degradation. The slightly higher rate in window 1 is due to buffered samples from initialization.

5.1.5 LIS2DS12 Oscillator Characteristics

The LIS2DS12 accelerometer uses an internal RC oscillator to drive the analog-to-digital converter. Unlike crystal-based oscillators, RC oscillators have inherent frequency variation:

- The 1600 Hz ODR setting is a *nominal* value based on a typical RC frequency.
- The actual frequency depends on process variation, supply voltage, and temperature.
- Per the LIS2DS12 datasheet (Table 3), the ODR tolerance is not tightly specified – the RC oscillator frequency can vary by up to 10% from nominal.
- The measured 1 493 Hz (-6.7% from nominal) is within the expected range and consistent across all four sensors on the module.

This variation is a fixed characteristic of each individual sensor and does not affect measurement quality. The sample rate remains stable over time, and no samples are lost (0 overruns). For applications requiring precise timing, the host-side timestamp from the acquisition script should be used rather than assuming an exact 1600 Hz rate.

6 Hardware

6.1 Wiring

The AF10R1 has a 4-pin connector for power and RS485 communication.

Pin	Name	Description
1	VIN	Supply voltage, 5V
2	GND	Ground
3	A	RS485 bus A (+)
4	B	RS485 bus B (-)

6.2 P100 Connector Pinout

P100 (Würth 653004114822) is the main 4-pin connector for power and RS485 communication, located on the left edge of the board.

Pin	Name	Net	Description
1	VIN	G	Supply voltage, 5V (power protection)
2	GND	GND	Ground
3	A	RS485_A	RS485 bus A (+)
4	B	RS485_B	RS485 bus B (-)

6.2.1 Pin 1 Identification

On the physical board, P100 is mounted on the left edge. Pin 1 is the topmost pin, closest to the top-left corner of the PCB, indicated by the arrow in the figure below. Pins are numbered sequentially downward (1 at top, 4 at bottom).

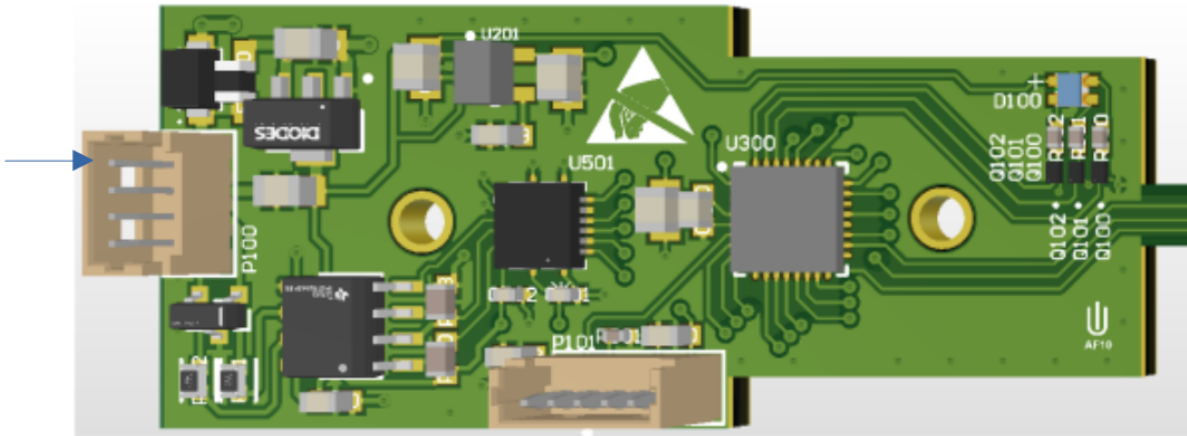
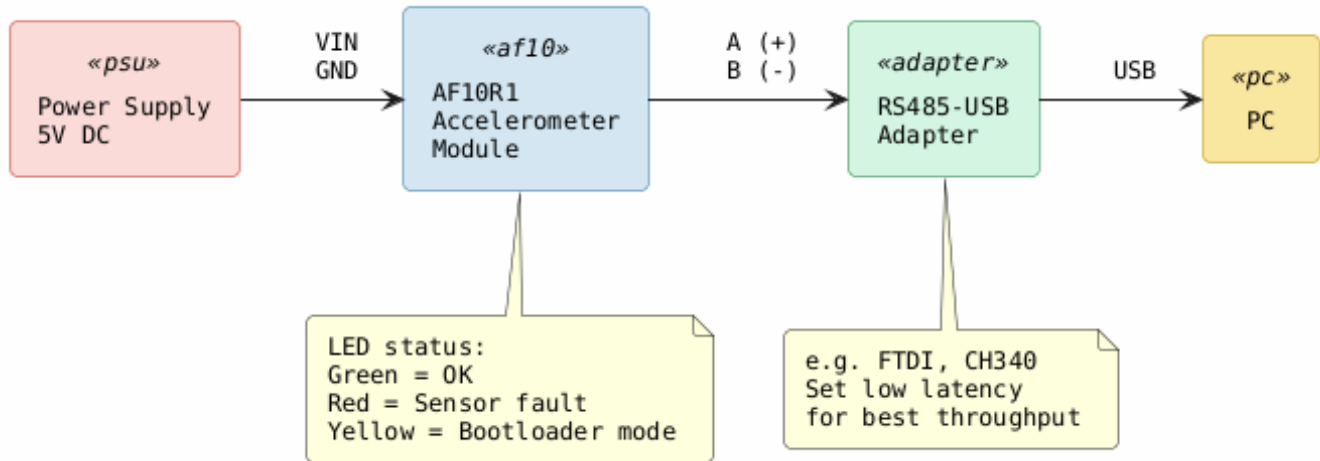


Figure 1: AF10R1 3D view. Arrow indicates pin 1 on P100.

6.3 Connection Diagram



6.4 Wiring Guidelines

- Keep RS485 cable length under 100 m for 921600 baud.
- Use twisted pair for A/B lines.
- Add 120 ohm termination resistor between A and B at each end of the bus if cable length exceeds 10 m.
- Do not exceed 24V on VIN.

6.5 Status LED

Color	Meaning
Green	Normal operation, all 4 sensors detected
Red	Sensor fault (at least one sensor not responding)
Yellow	Bootloader mode (waiting for firmware update)
Blue	Boot sequence (brief flash during startup)

6.6 Boot Sequence

1. LED test: red, green, blue (500 ms each).
2. Initialize 4 accelerometers.
3. LED turns green (all OK) or red (sensor fault).
4. Ready for Modbus commands.

7 Firmware Update

The AF10R1 includes a bootloader that allows firmware updates over the RS485 connection using the same Modbus interface. No physical access to debug connectors (SWD/JTAG) is required.

7.1 Requirements

- Python 3 with pyserial: `pip install pyserial`
- RS485 USB adapter connected to the AF10R1
- New firmware binary file (`.bin`)

7.2 Update Procedure

7.2.1 Step 1: Connect

Connect the RS485 USB adapter to the AF10R1 and identify the serial port (e.g. `/dev/ttyUSB0` on Linux, COM3 on Windows).

Verify that the unit is responding:

```
python3 flash_modbus.py /dev/ttyUSB0 --help
```

7.2.2 Step 2: Flash new firmware

Run the update script with the new firmware binary:

```
python3 flash_modbus.py /dev/ttyUSB0 new_firmware.bin
```

The script will:

1. Send a command to the running application to enter bootloader mode.
2. Wait for the bootloader to start (LED turns yellow).
3. Erase the application flash.
4. Transfer the new firmware in 256-byte blocks.
5. Validate the firmware image (header and CRC32 check).
6. Boot the new firmware.

Progress is shown during the transfer:

```
Bootloader v1.0 connected
Erasing application flash...
Erase complete.
```

```
Flashing 140 blocks...
```

```
[#####] 100% (140/140)
```

Flash write complete.

Validating image CRC32...
Image valid!

Booting application...
Done.

7.2.3 Step 3: Verify

After the update completes, the unit reboots into the new firmware. The LED should turn green, indicating normal operation. Verify by reading sensor data over Modbus.

7.3 LED Indicators During Update

Color	Meaning
Yellow	Bootloader active, waiting for firmware
Green (brief)	Image validated, booting application
Green (steady)	Application running normally

7.4 Troubleshooting

7.4.1 Update script reports “No response from bootloader”

- Verify the RS485 connection and correct serial port.
- Power cycle the unit and retry.
- If the unit is already in bootloader mode (yellow LED), add the `--skip-enter-bl` flag:

```
python3 flash_modbus.py /dev/ttyUSB0 firmware.bin --skip-enter-bl
```

7.4.2 Unit is unresponsive after a failed update

If an update was interrupted or the firmware is corrupted, the bootloader will remain in update mode (yellow LED). Simply run the update script again with `--skip-enter-bl`:

```
python3 flash_modbus.py /dev/ttyUSB0 firmware.bin --skip-enter-bl
```

The bootloader is protected and cannot be overwritten by the update process.

7.4.3 Validation fails

The script checks firmware integrity automatically. If validation fails, the unit stays in bootloader mode (yellow LED) and does not boot the corrupted image. Retry the update with a known-good firmware file.